事务

事务的四大原则ACID

1. A——（Atomicity）原子性：

   > 把多个SQL打包成一个整体，不可分割，要么执行全部，要么不执行

2. C——（Consistency）一致性：

   > 事务执行前与执行后，数据符合正常的逻辑的情况，如小明给小红转账100，事务执行后小明兜里少100块，小红兜里多100块

3. 💫I——（Isolation）隔离性：

   > 一个数据库中可以执行多个事务，并且要求使多个事务之间互相影响尽量小，故有隔离性概念

	事务的隔离级别		不同隔离级别产生的问题
隔离性低　效率高	1. read-uncommitted 读未提交	⇨	1. 脏读 + 不可重复读+幻读
	2. read-committed 读已提交	⇨	2. 不可重复读 + 幻读
隔离性高　效率低	3. repeatable read 重复读	⇨	3.幻读
	4. serializable 串行化		

1. read-uncommitted

事务1	事务2
begin	begin
xxxxxxx	xxxx（读到事务1操作的值）
commit	commit

2. read-committed

事务1	事务2
begin	begin
xxxxx	
commit	
	xxxxx
	commit

3. repeatable-read 解决 “不可重复读” 问题

不可重复读

事务1	事务2	事务3
begin		begin
修改小红总分为100		
commit		
	begin	读取小红总分=>100
	修改小红总分为90分	
	commit	
		读取小红总分=>90
		commit

4. serializable

事务1	事务2	事务3
begin		
xxxx		
commit		
	begin	
	xxx	
	commit	
		begin
		xxx
		commit

不同级别的隔离会产生不同的问题

1）脏读

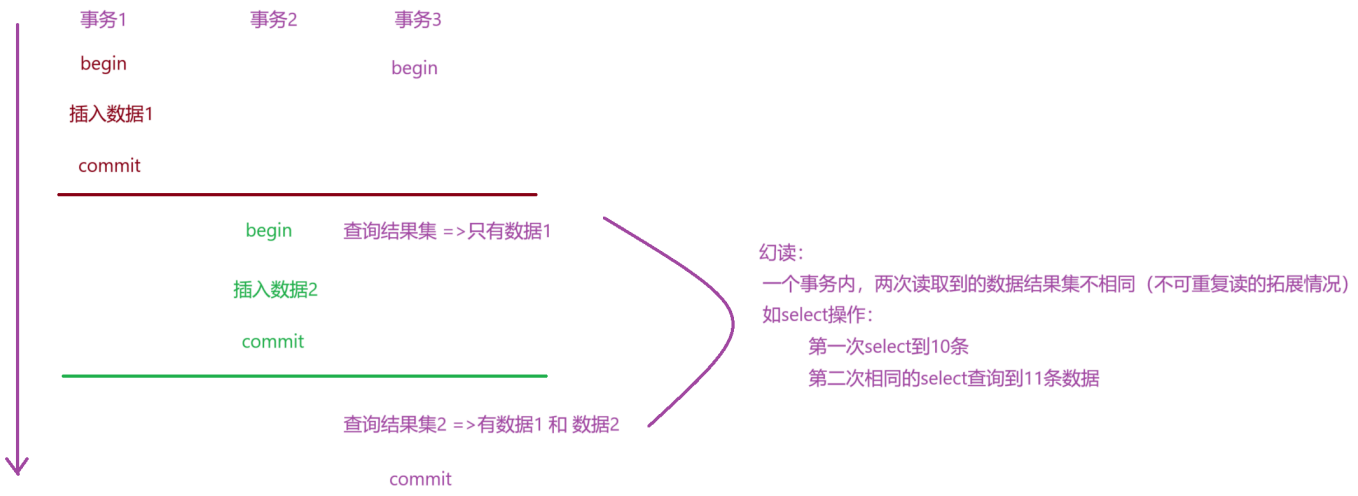
2）不可重复读

3）幻读

1）脏读　无效的数据

2）不可重复读

3）幻读



4. D——（Duration）持久性：

> 事务的执行的结果会保存在硬盘上

它与多线程中的锁[1]密切相关，其背后的逻辑是相似的

---

1. # 锁

   锁的存在就是防止线程冲突的情况，一个线程得到了锁，其他的线程如果想要对同一个变量进行修改，必须要等 上一个得到锁的线程释放锁后，其他的线程通过“公平竞争[2]”得到锁，才能往下执行，不然就只能等，进入阻塞状态

   事务

   事务的背后跟锁是密切相关的

锁的特点

1. 互斥 / 独占

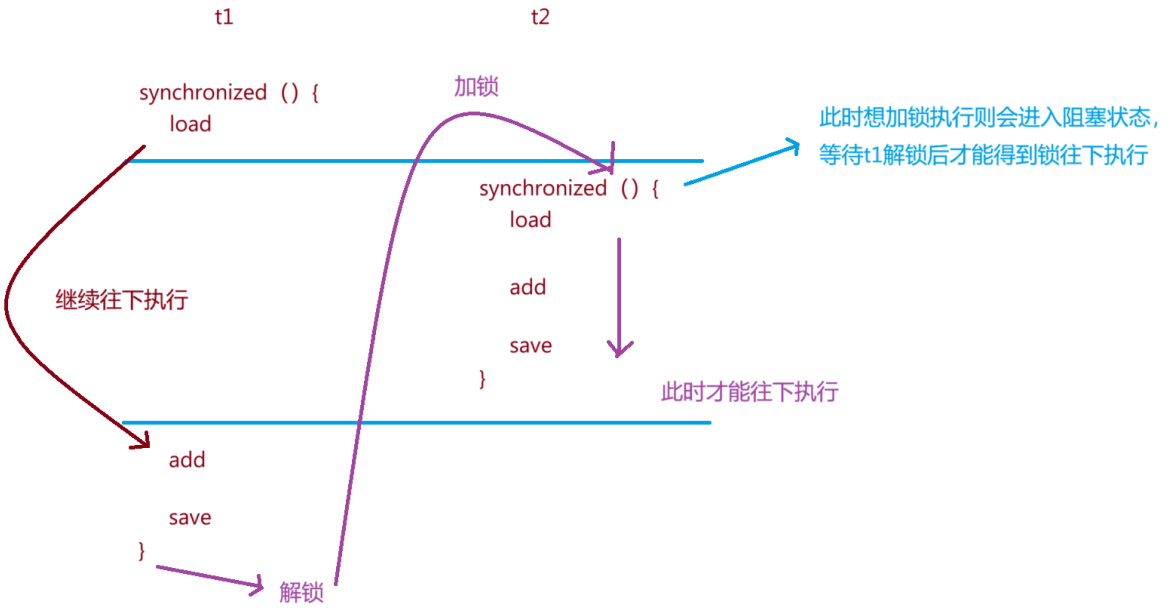
   > 有锁就会有线程的 锁竞争 / 锁冲突
   >
   > 其他语言中，上锁、解锁是由 lock（）与 unlock（）配合，但有个缺陷，可能写到最后就忘记解锁了，导致其他线程得不到锁无法执行下去，永远阻塞等待，也叫线程饿死

在Java中为了避免这种情况发生，直接规定了synchronize（locker）{ } 需要加锁的逻辑写到括号内，逻辑执行到括号外自动解锁，进入synchronized语句中自动上锁

- locker 的对象可以是任意的，只要是Object / Object子类的实例。但必须是多个线程对同一个对象进行加锁！！！！

  就像追妹子一样，多个老铁追同一个妹子才能产生锁🔒竞争的情况，要是一人各追一个，加锁就没有意义了~~毕竟线程竞争的本质是多个线程同时访问同一份共享资源产生争夺

- 如果其他的线程也尝试加锁操作就会产生阻塞，从而避免线程执行指令的过程中“被插队”



2. 锁的粒度

   锁内执行的逻辑多，粒度就大；相反代码逻辑少，粒度小

   这里的代码逻辑少不代表代码的行数，而是代码的执行逻辑，注意区分🎗️

3. 锁的开销

锁虽然好，但不是什么时候都需要加锁的，加锁解锁是需要消耗资源的，不能无脑加锁，加锁的代价使得程序的效率更低~~

4. 可重入锁

形如这样的代码

```
synchronized(locker){
    c.add();
}
public synchronized void add(){
    count++;
}
```

线程对c进行加锁，但add方法中又有一层锁，对c进行了二次加锁，这会产生问题吗？？

——Java已经帮我们解决了这个问题

理论上执行到第二个加锁的地方是会堵塞住无法继续往下执行的，但Java的synchronized做了特殊处理，对可重入锁规定了——第一次加锁成功后，synchronized内部会判定第二次加锁：如果是同一个线程加锁，则会跳过；如果是不同的线程加锁，则加锁生效，阻塞效果~~

5. 死锁

一个图了解死锁

- 两者都在等待对方释放锁，无限等待下去，形成了循环依赖的局面，造成了死锁
- 还有西方的哲学餐问题（n个线程m把锁）

平时写代码的时候很容易就写成死锁的情况，要千万注意注意再注意！！！！

死锁很重要

如何避免写出死锁的代码？我觉得首先要懂得形成死锁的原因

形成死锁的原因

只要打破其中一个就能避免死锁~~

- 锁是互斥的（锁的特性）
- 锁不可被强行抢占

以上两点是synchronized的特性，改变不了的。所以我们针对下面两个情况着重讨论

- 循环依赖请求

  如上我举的例子。

- 请求和保持

  如线程A获得了锁，但在锁内部中又请求了第二把锁，属于吃着碗里的 想着锅里的

  与上一个循环依赖请求有密切关系，如

```
Thread t1 = new Thread(() -> {
     synchronized (locker) {
          System.out.println("T1线程获得了锁");
          synchronized (locker2) {
               System.out.println("t1线程获得了第二把锁");
          }
      }
});
    t1.start();

单独运行这一个线程是没有问题的，但如果我们加多一个，同样是吃着碗里的想着锅里的线程呢?
```

```
Thread t1 = new Thread(() -> {
     synchronized (locker) {
         System.out.println("T1线程获得了锁");
         synchronized (locker2) {
            System.out.println("t1线程获得了第二把锁");
         }
      }
});
```

```
Thread t2 = new Thread(() -> {
     synchronized (locker2) {
        System.out.println("T2线程获得了锁");
        synchronized (locker) {
            System.out.println("t2线程获得了第二把锁");
        }
     }
});
   t1.start();
   t2.start();

最后就导致死锁
```

```
T1线程获得了锁
T2线程获得了锁
```

如何避免死锁

1. 打破等待循环——写代码的时候避免锁的嵌套
2. 打破请求与保持——约定加锁的顺序，给锁进行编号，从小到大依次加锁

2. 公平竞争

操作系统对于线程的调度是复杂的，随机的，我们感知不到，也无法干预

操作系统的开发大佬们在多线程对锁的竞争方面制定了规则

因此引入了**公平竞争与不公平竞争**

公平竞争vs不公平竞争

公平竞争

- 他们规定了 多个线程处于阻塞状态等待锁的期间，采取先来后到的方式，即最先请求锁的线程会得到锁

不公平竞争

- 则是当一个线程释放锁后，其他的线程 各凭本事 争取到锁

当然，系统的调度是一个庞大复杂的系统，不同的进程会有不同的调度方式，这种锁竞争方式是灵活多变的，我们只需要知道有这个东西就可以了